

Part 3 – Query-Specific Indexing Plans

Music Recommender System

We recommend six indexes for the point and range queries (Q1–Q4, Q7, Q8). The two scan queries (Q5, Q6) get no new index, because they read every row and an index would not reduce that work. Primary keys are indexed automatically by PostgreSQL; foreign keys are not, so `tracks.album_id` and `tracks.genre_id` start without an index.

Summary of recommended indexes

Query	Type	Index	Kind
Q1 Track within an album	Point	<code>tracks (album_id, track_name)</code>	Composite B-tree
Q2 Album by title + date	Point	<code>album (album_name, release_date)</code>	Composite B-tree
Q7 Artist by name + country	Point	<code>artists (artist_name, artist_country)</code>	Composite B-tree
Q3 Popular tracks in a genre	Range	<code>tracks (genre_id, popularity)</code>	Composite B-tree
Q4 Top-rated albums by period	Range	<code>album (genre_id, release_date)</code>	Composite B-tree
Q8 Albums by release period	Range	<code>album (release_date)</code>	Single-column B-tree
Q5 Genre feature comparison	Scan	None	—
Q6 Artist review comparison	Scan	None	—

```
CREATE INDEX idx_tracks_album_name ON tracks (album_id, track_name);
CREATE INDEX idx_album_name_date ON album (album_name, release_date);
CREATE INDEX idx_artists_name_ctry ON artists (artist_name, artist_country);
CREATE INDEX idx_tracks_genre_pop ON tracks (genre_id, popularity);
CREATE INDEX idx_album_genre_date ON album (genre_id, release_date);
CREATE INDEX idx_album_release_date ON album (release_date);
```

Point queries

Q1 — Find a specific track within an album

Index: `CREATE INDEX idx_tracks_album_name ON tracks (album_id, track_name);`

- **Structure:** both filter columns are equality conditions, so a composite B-tree locates the row in a single lookup. `album_id` leads because it also supports lookups of all tracks on an album and serves as the index for the `album_id` foreign key.
- **Selectivity:** an album holds roughly 10–20 tracks, and the track name narrows that to one row. This very high selectivity is the ideal case for an index.
- **Joins:** `album`, `genre` and `track_feel` are joined on their primary keys, and the artist subquery uses the `track_artists` primary key (`track_id`, `artist_id`). These are already indexed, so `tracks` is the only table that needs a new index.

Q2 — Find an exact album by title and release date

Index: `CREATE INDEX idx_album_name_date ON album (album_name, release_date);`

- **Structure:** a composite index on both equality columns. `album_name` leads because it is the more selective column, and searches by title alone can still use the index.
- **Selectivity:** a title plus a release date identifies a single album.
- **Joins:** `genre` and `album_scores` are joined on their primary keys, so no further index is needed.

Q7 — Find a specific artist

Index: `CREATE INDEX idx_artists_name_ctry ON artists (artist_name, artist_country);`

- **Structure:** a composite index on both equality columns.
- **Selectivity:** the artist name alone is nearly unique, and the country separates the rare artists who share a name. The query returns one row.

Range queries

Q3 — Find popular tracks within a selected genre

Index: `CREATE INDEX idx_tracks_genre_pop ON tracks (genre_id, popularity);`

- **Structure:** in a composite index, the equality column goes first and the range column second. All tracks for one genre sit together in the index, sorted by popularity, so only the requested popularity slice is read.
- **Sorting:** because the index is already ordered by popularity, the `ORDER BY popularity DESC` needs no separate sort step.
- **Selectivity:** one genre combined with a narrow popularity band returns a small fraction of the table. Reversing the columns to `(popularity, genre_id)` would force a read of every genre's tracks in the band.

Q4 — Find highly rated albums released during a selected period

Index: `CREATE INDEX idx_album_genre_date ON album (genre_id, release_date);`

- **Structure:** the same equality-then-range rule as Q3: `genre_id` first, `release_date` second. The index finds one genre's albums in the date window, which are then joined to `album_scores` by primary key.
- **Not indexed:** `metacritic_critic_score >= 80` is applied after the join. A threshold like this matches a large share of reviewed albums, so it is not selective enough to justify its own index.

Q8 — Find albums released during a selected period

Index: `CREATE INDEX idx_album_release_date ON album (release_date);`

- **Structure:** a single-column index, because the date range is the only filter. Results come back already in date order, so the `ORDER BY` costs nothing extra.

- **Why Q4's index cannot be reused:** (`genre_id`, `release_date`) cannot serve a date-only search, because its leading column is `genre_id`.
- **Selectivity:** the index pays off for narrow ranges (a month or a year). For very wide ranges that cover most of the table, the optimizer is expected to choose a full scan instead, because reading the table directly becomes cheaper.

Scan queries

Q5 — Compare the musical characteristics of different genres

Index: none recommended.

- The query reads every row of `tracks` and `track_feel` to compute averages per genre. With no filter, a sequential scan with a hash join is already the cheapest plan, and an index would only add extra lookups.

Q6 — Compare artists based on album review scores

Index: none recommended.

- The `artist_albums` step reads all of `track_artists` and `tracks`, then joins and groups the full result. No index removes this work; hash joins over full scans are the efficient plan for a whole-table aggregate.

Impact on inserts, updates and deletes

The six indexes are worth their write cost because a music catalog is read far more often than it changes. The main risk is updates to `tracks.popularity`.

Table	New indexes	Write pattern	Impact
tracks	2	Batch loads of new releases; popularity refreshed often	Medium: each insert updates 2 more indexes, and each popularity change updates <code>idx_tracks_genre_pop</code>
album	3	Batch loads; rare edits	Low: few writes, so 3 indexes add little cost
artists	1	Rare inserts; popularity refreshed often	Low: <code>artist_popularity</code> is not indexed, so its updates stay cheap

- **Inserts:** every index adds a write per new row. For large catalog loads, dropping the indexes, loading, and recreating them is faster than maintaining them row by row.
- **Updates:** because `popularity` is part of `idx_tracks_genre_pop`, each popularity change also rewrites an index entry and prevents PostgreSQL's cheaper in-place (HOT) update. This is acceptable for a nightly batch refresh but should be re-evaluated if popularity becomes real-time.
- **Deletes:** removing a row also removes its index entries. Deletes are rare in a catalog, so the cost is minor.
- **Storage:** each index uses additional disk space, which grows with the table.